

Problem Statement: Universal Database Editor

Project Title

Universal DB Editor (SQLite, PostgreSQL, MongoDB)

Background

Developers, analysts, and support engineers frequently work with multiple database technologies. Existing database management tools are often database-specific, expensive, overly complex, or require separate installations.

The goal of this project is to build a web-based database editor that allows users to connect to different database systems through a single interface and perform common database management tasks.

This project is intended as an onboarding assignment to evaluate an intern's ability to:

- Design scalable software architecture
 - Work with multiple database technologies
 - Build secure backend APIs
 - Develop intuitive frontend interfaces
 - Handle real-world edge cases
 - Write maintainable code
-

Objective

Build a browser-based database editor supporting:

1. SQLite
2. PostgreSQL
3. MongoDB

The application should allow users to connect to databases from:

- Local database files

- Uploaded files
 - Direct connection strings/URLs
 - Remote database servers
-

Core Requirements

1. Database Connections

SQLite

Support:

- Upload `.db`, `.sqlite`, `.sqlite3` files
- Open local files
- Persist connection information

Example:

inventory.db
users.sqlite

PostgreSQL

Support:

Host
Port
Database Name
Username
Password

Or connection string:

postgresql://user:password@localhost:5432/mydb

MongoDB

Support:

mongodb://localhost:27017

and

mongodb+srv://...

2. Connection Manager

Users should be able to:

- Create connection
- Edit connection
- Delete connection
- Favorite connection
- Reconnect

Display:

My SQLite DB
Production Postgres
Mongo Sandbox

3. Database Explorer

SQLite / PostgreSQL

Display hierarchy:

Database
├── Tables
│ ├── users
│ ├── orders
│ └── products
├── Views
└── Indexes

MongoDB

Display hierarchy:

Database
├── users

4. Data Viewer

Users can:

- Open table/collection
- Browse records
- Paginate records
- Sort records
- Search records
- Filter records

Example:

id	name	email
1	John	john@mail.com

5. Record Editor

Create Record

Add new row/document.

Update Record

Inline editing support.

Delete Record

Single or multiple records.

6. Schema Viewer

SQL

Show:

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY,  
  name TEXT,  
  email TEXT  
);
```

Mongo

Show collection metadata and indexes.

7. Query Playground

PostgreSQL / SQLite

Execute:

```
SELECT * FROM users;
```

Features:

- Syntax highlighting
- Query history
- Result grid
- Export results

MongoDB

Execute:

```
db.users.find({})
```

or

```
db.users.aggregate([])
```

8. CRUD Operations

Support:

Operation	SQLite	PostgreSQL	MongoDB
Create	Yes	Yes	Yes
Read	Yes	Yes	Yes
Update	Yes	Yes	Yes
Delete	Yes	Yes	Yes

9. Import / Export

Export

Support:

- CSV
- JSON
- SQL Dump

Import

Support:

- CSV
 - JSON
-

10. User Interface

Layout

Sidebar	Main Content Area	
Tables	Query Editor	
Collections	Result Grid	

Responsive support required.

Advanced Requirements

11. Visual Query Builder

Non-technical users should be able to:

Select Table
Choose Columns
Add Filters
Generate Query

without writing SQL.

12. Relationship Viewer

For SQL databases:

Display:

Users
|
Orders
|
Payments

using ER diagrams.

13. Saved Queries

Users can:

Save Query
Rename Query
Delete Query
Run Query

14. Database Comparison

Compare:

Table A
vs
Table B

or

Database A
vs
Database B

Show schema differences.

15. AI Assisted Query Generator (Bonus)

Input:

Show top 10 customers by revenue

Output:

SELECT ...

or Mongo query equivalent.

Security Requirements

Mandatory:

Prevent Dangerous Queries

Examples:

DROP DATABASE
TRUNCATE
DELETE *

Require confirmation dialogs.

Credential Handling

- Encrypt saved credentials
- Never expose passwords to frontend

Validation

- Validate connection strings
 - Prevent injection attacks
-

Non-Functional Requirements

Performance

- Load 100k+ rows with pagination
- Server-side filtering

Reliability

- Graceful connection failure handling
- Auto reconnect support

Scalability

- Plugin architecture for future databases

Example:

MySQL

MariaDB

Redis

ElasticSearch

can be added later.

Suggested Tech Stack

Frontend

- React
- Next.js
- TypeScript
- Tailwind CSS
- AG Grid / TanStack Table

Backend

- Node.js
- Express / NestJS

Database Drivers

SQLite:

- sqlite3
- better-sqlite3

PostgreSQL:

- pg

MongoDB:

- mongodb
-

Deliverables

- 1. Source Code
 - 2. README
 - 3. Architecture Diagram
 - 4. API Documentation
 - 5. Database Connector Layer
 - 6. Unit Tests
 - 7. Docker Setup
-

Evaluation Criteria

Area	Weight
Architecture	20%
Code Quality	20%
Security	15%
UI/UX	15%
Database Handling	15%
Testing	10%
Documentation	5%

Stretch Goal (What separates excellent interns)

Implement a provider-based architecture:

DatabaseProvider



so adding a new database requires implementing only a new provider and registering it in the system.

This demonstrates understanding of abstraction, extensibility, and production-grade software design rather than simply building a CRUD application.